PHASE 4 OF 18

Strings, StringBuilder & Regex

String creation and the String Constant Pool, immutability, comparison, the String API, formatting, text blocks, StringBuilder/StringBuffer, regular expressions and classic string programs

IN THIS PHASE

4.1 What Is a String?
4.2 Creating Strings and the String Constant Pool
4.3 Immutability
4.4 Comparing Strings
4.5 The String API
4.6 Formatting Strings
4.7 Text Blocks (Java 15+)
4.8 StringBuilder and StringBuffer
4.9 StringJoiner and String.join
4.10 Regular Expressions (java.util.regex)
4.11 Classic String Programs
4.12 Predict the Output
4.13 Interview Questions
4.14 Quick Revision

4.1 What Is a String?

A `String` is an **object** that represents an immutable sequence of characters. Three classes in `java.lang` (no import needed) work with text:

Class	Mutable?	Thread-safe?	Since	Use when
`String`	No (immutable)	Yes (immutability makes it safe)	1.0	Text that doesn't change: names, keys, messages
`StringBuffer`	Yes	Yes (synchronized methods)	1.0	Building text shared across threads (rare today)
`StringBuilder`	Yes	No	1.5	Building or modifying text in one thread (the usual choice)

All three implement the `CharSequence` interface, which provides read-only access through `length()`, `charAt()`, `subSequence()` and `chars()`. Methods that accept a `CharSequence`, such as `contains()`, `append()` and `String.join()`, work with any of them.

```
String name = "Akshit";
String one  = "A";                                  // a single character is fine too
char[] data = {'a', 'b', 'c'};
String fromChars = new String(data);                // "abc"
String fromBytes = new String(bytes, StandardCharsets.UTF_8);  // always pass a charset
```

NOTE

Before Java 9, a String stored its text in a `char[]` (2 bytes per character). Since Java 9, **compact strings** store a `byte[]` plus an encoding flag: Latin-1 text uses 1 byte per character, otherwise UTF-16. This roughly halves memory for typical English text.

4.2 Creating Strings and the String Constant Pool

Approach	Example	Where the object goes
String literal	`String s = "Java";`	**String Constant Pool (SCP)**, reused if already present
`new` keyword	`String s = new String("Java");`	A **new** object on the heap every time (plus the literal in the SCP)

The **String Constant Pool** is a special area **inside the heap** (since Java 7; it was in PermGen before). Before creating a literal, the JVM checks the pool: if an equal String exists, the reference is reused, so the pool never holds duplicates.

```
String s1 = "Java";
String s2 = "Java";
String s3 = new String("Java");

                HEAP
   +-----------------------------------------+
   |  String Constant Pool (SCP)             |
   |  +-------------------------------------+  |
   |  |   "Java"   <----  s1, s2 (shared)    |  |
   |  +-------------------------------------+  |
   |                                         |
   |   "Java"   <----  s3 (separate object)     |
   +-----------------------------------------+

s1 == s2  -> true     s1 == s3  -> false     s1.equals(s3) -> true
```

How many objects are created?

Statement (run in order)	New objects	Why
`String s1 = "Hello";`	1	Literal placed in the SCP
`String s2 = "Hello";`	0	Reuses the SCP object
`String s3 = new String("Hello");`	1	Heap object only; `"Hello"` is already pooled
`String s4 = new String("Bye");`	2	`"Bye"` literal in the SCP + heap object
`String s5 = "Hel" + "lo";`	0	Constant expression folded at compile time to `"Hello"`

> **INTERVIEW**
> "How many objects does `new String("Hello")` create?" The standard answer is **two** (one in the pool for the literal, one on the heap) if the literal isn't pooled yet, otherwise **one**.

Benefits of the pool

- **Memory:** one shared copy of each repeated literal.
- **Performance:** fewer allocations, and `==` comparison of pooled constants is cheap.
- **Safe sharing:** possible only because Strings are immutable.

4.3 Immutability

Once created, a String object's content **can never change**. Every "modifying" method returns a **new** String.

```
String s = "Hello";
s.concat(" Java");          // creates "Hello Java" but the result is discarded
System.out.println(s);      // Hello  (unchanged!)

s = s.concat(" Java");      // reassign the REFERENCE to the new object
System.out.println(s);      // Hello Java   ("Hello" itself still exists unchanged)

s.toUpperCase();            // same mistake: result ignored
s = s.toUpperCase();        // correct
```

Why Java made String immutable

1. **String pool:** many references share one object. If one could change it, every other reference would see the change.
2. **Security:** class names, file paths, URLs, DB credentials and usernames are Strings. They can't be changed after they have been validated.
3. **Thread safety:** immutable objects can be shared between threads without synchronization.
4. **Hash code caching:** the hash is computed once and cached, which makes Strings fast and reliable as `HashMap` keys.
5. **Class loading:** class names passed to class loaders can't be tampered with.

CORRECTED FROM ORIGINAL NOTES

The original notes have the heading "Why Strings are **mutable**"; it should say **immutable**. Its example also needs clarifying: after `String s2 = s1; s1 = "User";`, `s2` is still `"Admin"` because only `s1`'s *reference* changed. Immutability matters because nobody can change the shared `"Admin"` object itself.

PITFALL

Immutability means a String's **content** can't change. The **variable** can still point to a different String unless it is `final`.

4.4 Comparing Strings

Way	Compares	Example	Result
`==`	**References** (same object?)	`"a" == new String("a")`	`false`
`equals()`	**Content**, case-sensitive	`"Java".equals("java")`	`false`
`equalsIgnoreCase()`	Content, ignoring case	`"JAVA".equalsIgnoreCase("java")`	`true`
`compareTo()`	Lexicographic (dictionary) order	`"apple".compareTo("banana")`	negative
`compareToIgnoreCase()`	Order, ignoring case	`"A".compareToIgnoreCase("a")`	`0`
`Objects.equals(a, b)`	Null-safe content check	`Objects.equals(null, null)`	`true`
`contentEquals()`	Content vs any `CharSequence`	`"ab".contentEquals(sb)`	`true` / `false`

```
String s1 = "Hello", s2 = "Hello";
String s3 = new String("Hello"), s4 = new String("Hello");
System.out.println(s1 == s2);         // true  (same pooled object)
System.out.println(s3 == s4);         // false (two heap objects)
System.out.println(s1 == s3);         // false
System.out.println(s3.equals(s4));    // true  (same content)

// compareTo: 0 if equal, negative if this < other, positive if this > other
"abc".compareTo("abc");     // 0
"a".compareTo("b");         // -1 ('a' - 'b')
"apple".compareTo("app");   // 2  (length difference when one is a prefix)
"Z".compareTo("a");         // -7 (uppercase letters sort BEFORE lowercase)
```

Compile-time constants and ==

```
String a = "Java";
String b = "Ja" + "va";        // folded at compile time -> same pooled "Java"
System.out.println(a == b);    // true

String part = "Ja";
String c = part + "va";        // built at RUNTIME -> new heap object
System.out.println(a == c);    // false

final String fpart = "Ja";
String d = fpart + "va";       // final constant -> folded at compile time
System.out.println(a == d);    // true
```

intern()

`intern()` returns the **pooled** copy of a String. If none exists yet, it adds this one to the pool first.

```
String s1 = new String("Hello");   // heap object
String s2 = s1.intern();           // reference to the pooled "Hello"
String s3 = "Hello";
System.out.println(s1 == s2);      // false (heap vs pool)
System.out.println(s2 == s3);      // true  (both pooled)
```

> **REMEMBER**
> Always use `equals()` to compare String content; `==` is almost always a bug. For null-safety, put the literal first (`"ADMIN".equals(role)`) or use `Objects.equals(a, b)`.

4.5 The String API

Indexes start at **0**. Ranges are **[begin, end)**: begin is inclusive, end is exclusive.

```
String s = "Hello";      index:  0   1   2   3   4
                         char:   H   e   l   l   o      s.length() = 5
```

Category	Method	Example	Result
Inspect	`length()`	`"Hello".length()`	`5`
Inspect	`charAt(i)`	`"Hello".charAt(1)`	`'e'`
Inspect	`isEmpty()`	`"".isEmpty()`	`true`
Inspect	`isBlank()` (Java 11)	`" ".isBlank()`	`true`
Search	`indexOf(str)` / `indexOf(str, from)`	`"banana".indexOf("an")`	`1`
Search	`lastIndexOf(str)`	`"banana".lastIndexOf('a')`	`5` (`-1` if absent)
Search	`contains(seq)`	`"Hello".contains("ell")`	`true`
Search	`startsWith()` / `endsWith()`	`"Hello".startsWith("He")`	`true`
Search	`matches(regex)`	`"12345".matches("\\d+")`	`true` (whole string)
Extract	`substring(begin)`	`"Hello".substring(2)`	`"llo"`
Extract	`substring(begin, end)`	`"Hello".substring(1, 4)`	`"ell"`
Extract	`toCharArray()`	`"abc".toCharArray()`	`['a','b','c']`
Extract	`chars()`	`"abc".chars().count()`	`3` (IntStream)
Transform	`toUpperCase()` / `toLowerCase()`	`"java".toUpperCase()`	`"JAVA"`
Transform	`trim()`	`" Java ".trim()`	`"Java"` (removes chars ≤ space)

Category	Method	Example	Result
Transform	`strip()`, `stripLeading()`, `stripTrailing()` (Java 11)	`"\u2003Java ".strip()`	`"Java"` (Unicode-aware)
Transform	`replace(old, new)`	`"Java".replace('a', 'o')`	`"Jovo"` (literal, all occurrences)
Transform	`replaceAll(regex, rep)` / `replaceFirst()`	`"a1b22".replaceAll("\\d", "")`	`"ab"`
Transform	`concat(str)`	`"Hi".concat(" there")`	`"Hi there"`
Transform	`repeat(n)` (Java 11)	`"ab".repeat(3)`	`"ababab"`
Split / join	`split(regex)`	`"A,B,C".split(",")`	`[A, B, C]`
Split / join	`String.join(delim, parts)`	`String.join("-", "a", "b")`	`"a-b"`
Split / join	`lines()` (Java 11)	`"a\nb".lines().count()`	`2`
Compare	`equals`, `equalsIgnoreCase`, `compareTo`	see the previous section	
Convert	`String.valueOf(x)`	`String.valueOf(10.5)`	`"10.5"`
Format	`String.format()` / `formatted()` (Java 15)	`"%d%%".formatted(5)`	`"5%"`
Pool	`intern()`	`s.intern()`	pooled reference

```
String str = "Hello", str1 = "Java", str2 = "JAVA", str3 = "  Java  ";
System.out.println(str.charAt(1));                    // e
System.out.println(str1.equalsIgnoreCase(str2));      // true
System.out.println(str.substring(1, 4));              // ell
System.out.println(str3.trim() + "|");                // Java|
System.out.println(Arrays.toString("A,B,C".split(","))); // [A, B, C]
System.out.println("banana".replace("an", "AN"));     // bANANa
System.out.println("  ".isEmpty() + " " + "  ".isBlank());  // false true

"Hello".charAt(10);        // StringIndexOutOfBoundsException
"Hello".substring(-1, 4);  // StringIndexOutOfBoundsException
"Hello".substring(3, 1);   // StringIndexOutOfBoundsException (begin > end)
"Hello".substring(5);      // "" (begin == length is allowed)
```

split() gotchas

```
"a.b.c".split(".");          // [] : '.' is a regex meaning "any character"!
"a.b.c".split("\\.");        // [a, b, c]   escape regex metacharacters
"a|b".split("\\|");          // [a, b]
"a.b".split(Pattern.quote("."));   // [a, b]  quote() treats the text literally

"a,b,,c,,".split(",");       // [a, b, , c]  trailing empty strings REMOVED
"a,b,,c,,".split(",", -1);   // [a, b, , c, , ]  limit -1 keeps them (CSV parsing!)
"a,b,c".split(",", 2);       // [a, b,c]     at most 2 parts
"  Hello   Java  ".trim().split("\\s+");   // [Hello, Java]  any run of whitespace
```

> **TIP**
> Regex metacharacters that need escaping in `split()`/`replaceAll()` are `. $ | ( ) [ ] { } ^ ? * + \`.
> Use `replace()` (not `replaceAll()`) for plain-text replacement: it is literal and needs no escaping.

4.6 Formatting Strings

```
String msg = String.format("Account %s has balance Rs.%,.2f", "SB-1001", 125000.5);
// Account SB-1001 has balance Rs.125,000.50
String msg2 = "Hi %s, you are %d".formatted("Rohit", 35);   // Java 15+
System.out.printf("%-10s|%5d|%n", "Item", 42);              // left/right aligned columns
```

Specifier	Meaning	Example	Output
`%s`	String (any object via `toString()`)	`format("%s", "hi")`	`hi`
`%d`	Integer	`format("%d", 42)`	`42`
`%f` / `%.2f`	Floating point / 2 decimals	`format("%.2f", 3.14159)`	`3.14`
`%,d`	Thousands separator	`format("%,d", 1000000)`	`1,000,000`
`%05d`	Zero-padded width 5	`format("%05d", 42)`	`00042`
`%10s` / `%-10s`	Width 10, right / left aligned	`format("[%-5s]", "ab")`	`[ab ]`
`%x` / `%X`	Hexadecimal	`format("%x", 255)`	`ff`
`%c`, `%b`, `%e`	Char, boolean, scientific	`format("%e", 12345.0)`	`1.234500e+04`
`%n`, `%%`	Platform newline, literal %	`format("%d%%", 5)`	`5%`

4.7 Text Blocks (Java 15+)

Multi-line string literals with no `\n` or escaped quotes. Ideal for JSON, SQL, XML and HTML in code and tests.

```
String json = """
        {
          "accountId": "SB-1001",
          "status": "ACTIVE"
        }
        """;          // the indentation of the closing """ sets the left margin

String sql = """
        SELECT id, name
        FROM customers
        WHERE city = ?
        """;
```

4.8 StringBuilder and StringBuffer

Both are **mutable** character sequences. They modify the **same object** instead of creating new ones, which is much faster when text changes repeatedly. Their APIs are identical; the difference is synchronization.

```
StringBuilder sb = new StringBuilder("Hello");
sb.append(" Java");                 // Hello Java
sb.insert(5, ",");                  // Hello, Java
sb.replace(0, 5, "Hi");             // Hi, Java
sb.delete(2, 3);                    // Hi Java
sb.deleteCharAt(0);                 // i Java
sb.setCharAt(0, 'I');               // I Java
sb.reverse();                       // avaJ I
String result = sb.toString();      // convert back to an immutable String

String csv = new StringBuilder()    // methods return 'this' -> chaining
        .append("id").append(',').append(101).append(',').append(true)
        .toString();                // "id,101,true"

// StringBuilder sb2 = "Hello";     // COMPILE ERROR: a literal is a String, not a StringBuilder
```

Method	Purpose	Returns
`append(x)`	Adds any type at the end (overloaded for all primitives and objects)	same builder
`insert(i, x)`	Inserts at index `i`	same builder
`delete(start, end)` / `deleteCharAt(i)`	Removes a range (end exclusive) / one char	same builder
`replace(start, end, str)`	Replaces a range	same builder
`reverse()`	Reverses in place	same builder
`setCharAt(i, c)` / `setLength(n)`	Changes one char / truncates or pads	`void`
`charAt`, `indexOf`, `lastIndexOf`, `substring`, `length`	Same meaning as in String	various
`capacity()` / `ensureCapacity(n)` / `trimToSize()`	Internal buffer size management	`int` / `void`
`toString()`	New immutable String with the current content	`String`

Capacity

The default capacity is **16** characters. `new StringBuilder("Hello")` has capacity `16 + 5 = 21`. When full, the builder grows to `(old capacity × 2) + 2` and copies the content. If you know the final size, pre-size it: `new StringBuilder(1024)`.

```
StringBuilder sb = new StringBuilder();
System.out.println(sb.capacity());   // 16
sb.append("12345678901234567");      // 17 chars > 16
System.out.println(sb.capacity());   // 34 = 16 * 2 + 2
```

Comparison: String vs StringBuffer vs StringBuilder

Feature	String	StringBuffer	StringBuilder
Mutability	Immutable	Mutable	Mutable
Modification	Creates a new object	Changes the same object	Changes the same object
Thread-safe	Yes (immutable)	Yes (synchronized)	No
Speed for repeated changes	Slowest (creates garbage)	Medium (lock overhead)	Fastest
`equals()`	Compares content	**Not overridden** (reference)	**Not overridden** (reference)
Storage	SCP (literals) or heap	Heap	Heap
Since	1.0	1.0	1.5

```
// Why it matters: + inside a loop creates a new String (and garbage) every iteration
String s = "";
for (int i = 0; i < 10_000; i++) s += i;          // O(n^2) copying: slow

StringBuilder sb = new StringBuilder();
for (int i = 0; i < 10_000; i++) sb.append(i);    // amortised O(n): fast
```

PITFALL

`new StringBuilder("a").equals(new StringBuilder("a"))` is **false**, because StringBuilder and StringBuffer don't override `equals()`. Compare `sb1.toString().equals(sb2.toString())`, or use `sb1.compareTo(sb2) == 0` (Java 11+).

> **NOTE**
> For a **single** expression such as `"Hi " + name + "!"`, the compiler already optimizes concatenation (StringBuilder before Java 9; `invokedynamic` string concat since Java 9), so `+` is fine there. The cost appears with `+=` inside **loops**.

4.9 StringJoiner and String.join

```
String.join(", ", "Java", "Python", "Go");          // Java, Python, Go
String.join("/", List.of("home", "rohit", "docs")); // home/rohit/docs

StringJoiner sj = new StringJoiner(", ", "[", "]"); // delimiter, prefix, suffix
sj.add("A").add("B").add("C");
System.out.println(sj);                             // [A, B, C]

List.of("x", "y").stream().collect(Collectors.joining("|"));   // x|y
```

> **NOTE**
> The original short notes list the rules for writing your own **immutable class** (final class, private final fields, no setters, defensive copies). They are covered fully, with examples, in Phase 8.

4.10 Regular Expressions (java.util.regex)

A **regular expression (regex)** is a pattern that describes text. Java provides two main classes:

- `Pattern`: a *compiled* regex. Create it once with `Pattern.compile()` and reuse it; it is thread-safe.
- `Matcher`: applies a Pattern to a specific input string and tracks the matches. It is not thread-safe.

> **CORRECTED FROM ORIGINAL NOTES**
> In the original short notes, the descriptions of the `Pattern` and `Matcher` method tables are swapped. `compile()`, `matcher()`, `matches(regex, input)`, `split()` and `quote()` belong to **Pattern**. `find()`, `matches()`, `group()`, `start()`, `end()` and `replaceAll()` belong to **Matcher**.

```
import java.util.regex.*;

Pattern p = Pattern.compile("\\d+");          // one or more digits
Matcher m = p.matcher("The car costs $500, the bike costs $200.");
while (m.find()) {                            // find the NEXT match each call
    System.out.println(m.group() + " at " + m.start() + "-" + m.end());
}
// 500 at 15-18
// 200 at 36-39

Pattern.matches("\\d{6}", "560001");          // true: the WHOLE input must match
"560001".matches("\\d{6}");                   // same, via String
```

Pattern method	Purpose
`compile(regex)` / `compile(regex, flags)`	Compile a regex (optionally with flags)
`matcher(input)`	Create a Matcher for an input
`matches(regex, input)` (static)	One-off full match
`split(input)`	Split by the pattern
`quote(text)`	Escape text so it is matched literally

Matcher method	Purpose
`matches()`	Does the **entire** input match?
`find()`	Is there a **next** match anywhere? (use in a loop)

Matcher method	Purpose
`lookingAt()`	Does the **beginning** of the input match?
`group()` / `group(n)` / `group("name")`	Text of the whole match / group n / named group
`start()`, `end()`	Index range of the current match
`replaceAll(rep)`, `replaceFirst(rep)`	Replace matches (`$1` refers to group 1)
`reset()`	Start matching again from the beginning

Regex syntax cheat sheet

Syntax	Meaning	Syntax	Meaning
`.`	Any character (except newline)	`\d` / `\D`	Digit / non-digit
`[abc]`	One of a, b or c	`\w` / `\W`	Word char `[a-zA-Z0-9_]` / non-word
`[^abc]`	Any character except a, b, c	`\s` / `\S`	Whitespace / non-whitespace
`[a-z]`, `[A-Z0-9]`	Ranges	`\b`	Word boundary
`^` / `$`	Start / end of input (or line with MULTILINE)	`\.`	A literal dot
`X*`	0 or more	`X{n}`	Exactly n
`X+`	1 or more	`X{n,}`	At least n
`X?`	0 or 1 (optional)	`X{n,m}`	Between n and m
`X*?`, `X+?`	**Lazy** (as few as possible)	`a|b`	a or b
`(X)`	Capturing group	`(?<name>X)`	Named group
`(?:X)`	Non-capturing group	`(?i)`	Inline case-insensitive flag

> **REMEMBER**
> In Java source code the backslash must be escaped, so the regex `\d` is written `"\\d"` and a literal backslash is `"\\\\"`.

Flag	Effect
`Pattern.CASE_INSENSITIVE`	Ignore case (`(?i)` inline)
`Pattern.MULTILINE`	`^` and `$` match at line breaks
`Pattern.DOTALL`	`.` also matches newline
`Pattern.UNICODE_CASE`	Unicode-aware case folding (with CASE_INSENSITIVE)
`Pattern.COMMENTS`	Allow whitespace and `#` comments in the pattern

Practical patterns (validation and extraction)

Purpose	Regex (as a Java string)
10-digit Indian mobile starting 6–9	`"[6-9]\\d{9}"`
PIN code (6 digits, not starting 0)	`"[1-9]\\d{5}"`
PAN card	`"[A-Z]{5}[0-9]{4}[A-Z]"`
IFSC code	`"[A-Z]{4}0[A-Z0-9]{6}"`
Simple email	`"^[\\w.+-]+@[\\w-]+(\\.[\\w-]+)+$"`
Date `dd-MM-yyyy` (format only)	`"\\d{2}-\\d{2}-\\d{4}"`
Strong password (8+ chars, upper, lower, digit, special)	`"^(?=.*[a-z])(?=.*[A-Z])(?=.*\\d)(?=.*[@#$%!]).{8,}$"`

```java
// Pre-compile once (static final) and reuse: compiling is the expensive part
private static final Pattern PAN = Pattern.compile("[A-Z]{5}[0-9]{4}[A-Z]");
boolean ok = PAN.matcher("ABCDE1234F").matches();            // true

// Capturing groups: extract day, month, year
Matcher m = Pattern.compile("(\\d{2})-(\\d{2})-(\\d{4})").matcher("Due: 15-08-2026");
if (m.find()) {
    System.out.println(m.group(1) + "/" + m.group(2) + "/" + m.group(3)); // 15/08/2026
}

// Named groups
Matcher n = Pattern.compile("(?<code>[A-Z]{3})(?<amt>\\d+)").matcher("INR500");
if (n.matches()) System.out.println(n.group("code") + " " + n.group("amt")); // INR 500

// Replace using group references: mask all but the last 4 card digits
"4111111111111111".replaceAll("\\d(?=\\d{4})", "*");   // ************1111
"2026-08-15".replaceAll("(\\d{4})-(\\d{2})-(\\d{2})", "$3/$2/$1");  // 15/08/2026

// Greedy vs lazy
"<b>bold</b>".replaceAll("<.*>", "");    // ""     greedy .* eats everything
"<b>bold</b>".replaceAll("<.*?>", "");   // "bold" lazy .*? stops at the first >
```

PITFALL

`String.matches()`, `replaceAll()` and `split()` compile the regex **on every call**. In a loop or a hot path, use a pre-compiled `static final Pattern`. Also, `matches()` needs the **whole** string to match; use `find()` to search inside it.

4.11 Classic String Programs

```java
// 1. Reverse a string: three ways
String s = "Java";
String r1 = new StringBuilder(s).reverse().toString();   // "avaJ"  (best)
String r2 = "";
for (int i = s.length() - 1; i >= 0; i--) r2 += s.charAt(i);  // loop (fine for practice)
char[] c = s.toCharArray();                                   // two-pointer swap
for (int i = 0, j = c.length - 1; i < j; i++, j--) { char t = c[i]; c[i] = c[j]; c[j] = t; }
String r3 = new String(c);

// 2. Palindrome check (ignore case and non-letters)
static boolean isPalindrome(String str) {
    String t = str.toLowerCase().replaceAll("[^a-z0-9]", "");
    for (int i = 0, j = t.length() - 1; i < j; i++, j--)
        if (t.charAt(i) != t.charAt(j)) return false;
    return true;
}   // isPalindrome("Madam, I'm Adam") -> true
```

```java
// 3. Anagram: sort-based O(n log n), from the original notes
static boolean isAnagram(String a, String b) {
    char[] x = a.toLowerCase().toCharArray(), y = b.toLowerCase().toCharArray();
    if (x.length != y.length) return false;
    Arrays.sort(x); Arrays.sort(y);
    return Arrays.equals(x, y);
}
// 3b. Anagram: counting O(n) for lowercase a-z
static boolean isAnagramFast(String a, String b) {
    if (a.length() != b.length()) return false;
    int[] count = new int[26];
    for (int i = 0; i < a.length(); i++) {
        count[a.charAt(i) - 'a']++;
        count[b.charAt(i) - 'a']--;
    }
    for (int n : count) if (n != 0) return false;
    return true;
}   // listen/silent, heart/earth -> true;  hello/world -> false
```

```java
// 4. Count vowels / remove vowels
String text = "Hello Java Programming";
long vowels = text.toLowerCase().chars().filter(ch -> "aeiou".indexOf(ch) >= 0).count(); // 7
String noVowels = text.replaceAll("[aeiouAEIOU]", "");      // Hll Jv Prgrmmng

// 5. Word count (handles multiple spaces)
int words = text.isBlank() ? 0 : text.trim().split("\\s+").length;   // 3

// 6. Character frequency (insertion order kept)
Map<Character, Integer> freq = new LinkedHashMap<>();
for (char ch : "programming".toCharArray()) freq.merge(ch, 1, Integer::sum);
// {p=1, r=2, o=1, g=2, a=1, m=2, i=1, n=1}

// 7. First non-repeating character
for (Map.Entry<Character, Integer> e : freq.entrySet())
    if (e.getValue() == 1) { System.out.println(e.getKey()); break; }   // p
```

```java
// 8. Reverse the order of words
String[] parts = "I love Java".trim().split("\\s+");
Collections.reverse(Arrays.asList(parts));
String reversed = String.join(" ", parts);          // "Java love I"

// 9. Remove duplicate characters, keeping order
StringBuilder out = new StringBuilder();
for (char ch : "banana".toCharArray())
    if (out.indexOf(String.valueOf(ch)) == -1) out.append(ch);    // "ban"

// 10. Toggle case
StringBuilder tg = new StringBuilder();
for (char ch : "HeLLo".toCharArray())
    tg.append(Character.isUpperCase(ch) ? Character.toLowerCase(ch)
                                        : Character.toUpperCase(ch));   // "hEllO"

// 11. Contains only digits?
boolean digits = "12345".chars().allMatch(Character::isDigit);  // true
```

> **TIP**
>
> Useful `Character` helpers: `isDigit`, `isLetter`, `isLetterOrDigit`, `isWhitespace`, `isUpperCase`, `isLowerCase`, `toUpperCase`, `toLowerCase`, `getNumericValue`.

4.12 Predict the Output

```java
String s1 = "Java";
String s2 = s1;
s1 += " Rocks";
System.out.println(s2);                         // Java (s2 still refers to the old object)

String a = "Hi", b = new String("Hi");
System.out.println(a == b.intern());            // true

System.out.println("Java".substring(4));        // "" (empty, no exception)
System.out.println("abc".compareTo("abd"));     // -1
System.out.println("10" + 20 + 30);             // 102030
System.out.println(10 + 20 + "30");             // 3030
String n = null;
System.out.println(n + "x");                    // nullx (no NPE for concatenation)
System.out.println(n.length());                 // NullPointerException

StringBuilder sb = new StringBuilder("abc");
StringBuilder sb2 = sb.append("d");
System.out.println(sb == sb2);                  // true (append returns the same object)
System.out.println("a,b,,".split(",").length);  // 2 (trailing empties removed)
System.out.println("Hello".replace('l', 'L').indexOf('L'));   // 2
```

4.13 Interview Questions

Q. Why is String immutable in Java?
To make the String pool safe (shared literals can't be changed through one reference), for security (paths, URLs, class names and credentials can't be altered after validation), for thread safety, and so the hash code can be cached for fast `HashMap` lookups.

Q. What is the String Constant Pool?
A special area in the heap (since Java 7) where literals are stored without duplicates. A literal that already exists in the pool is reused instead of creating a new object.

Q. What is the difference between `==` and `equals()` for Strings?
`==` compares references (same object?). `equals()` compares character content. Use `equals()` for content.

Q. How many objects are created by `String s = new String("abc");`?
Up to two: the `"abc"` literal in the pool (if not already there) and a new String object on the heap.

Q. What does `intern()` do?
It returns the canonical pooled instance of the String, adding it to the pool if absent. Afterwards, `==` works between interned strings.

Q. What is the difference between String, StringBuffer and StringBuilder?
String is immutable. StringBuffer and StringBuilder are mutable. StringBuffer's methods are synchronized (thread-safe, slower); StringBuilder's are not (faster). Use StringBuilder for building strings in a single thread.

Q. Why should you avoid `+=` with Strings in a loop?
Each `+=` creates a new String and copies all previous characters, which is $O(n^2)$ and creates lots of garbage. StringBuilder appends into one growing buffer.

Q. Does StringBuilder override `equals()`?
No. It uses `Object.equals()` (reference comparison). Compare `toString()` results, or use `compareTo()` (Java 11+).

Q. What is the difference between `trim()` and `strip()`?
`trim()` removes characters with code ≤ U+0020 (ASCII whitespace and control characters). `strip()` (Java 11) removes all Unicode whitespace, such as U+2003 em space.

Q. What is the difference between `replace()` and `replaceAll()`?
`replace()` does literal replacement of all occurrences (char or CharSequence). `replaceAll()` treats its first argument as a **regex**.

Q. What is the difference between `Matcher.matches()` and `find()`?
`matches()` requires the entire input to match the pattern. `find()` searches for the next matching subsequence anywhere in the input and can be called repeatedly.

Q. Why is String a popular `HashMap` key?
It is immutable (its hash can't change after insertion), it caches its hash code, and it correctly overrides `equals()` and `hashCode()`.

4.14 Quick Revision

- String is an **immutable object**; StringBuilder/StringBuffer are **mutable**. All implement `CharSequence`.
- Literals go into the **String Constant Pool** (in the heap); `new String()` always creates a new heap object.
- `==` compares references; `equals()` **compares content**. Use a literal first or `Objects.equals` for null safety.

- Compile-time constants (`"Ja" + "va"`, `final` parts) are folded and pooled; runtime concatenation is not.
- String methods return **new** Strings, so assign the result: `s = s.trim();`.
- Index ranges are [begin, end); a bad index throws `StringIndexOutOfBoundsException`.
- `split`/`replaceAll`/`matches` take **regex**: escape `.` and `|`; `split` drops trailing empties unless limit is -1.
- Java 11: `isBlank`, `strip`, `repeat`, `lines`. Java 15: text blocks, `formatted()`.
- StringBuilder: default capacity 16, grows to `old*2 + 2`, does **not** override `equals()`.
- `StringBuffer` = synchronized; `StringBuilder` = fast. Never use `+=` in loops.
- Regex: `Pattern` (compiled, reusable, thread-safe) + `Matcher` (`find`, `matches`, `group`). Write `\d` as `"\\d"`.
- Pre-compile hot regexes as `static final Pattern`.